Ops — compiled path

> Compiled from `/home/dnikolic/Documents/Learning/Plan ucenja/paths/Ops`. Canonical sources stay in place.

Phase — 01-overview-and-scope

Directory: `01-overview-and-scope/`

Source: `01-overview-and-scope/01-program-architecture-thinking-and-roadmap.md`

Ops / Platform / Cloud — mindset and phased roadmap

This path supports **hands-on comprehension** of how modern backends are **built**, **tested**, **packaged**, **distributed**, **deployed**, **observed**, **maintained**, **incident-handled**, and **safely improved** — not trivia memorised separately from systemic behaviour.

What deep understanding means here

How an artefact survives from source control curiosity into repeatable production behaviours: pipelines, artefacts, rollout surfaces, observable runtime, corrective action, guarded evolution.

Area map (numeric order = topic progression)

#	Folder	Role
`01-*`	Overview	Ethos, practise rule, resource index
`02-*`	Linux administration + troubleshooting	Host triage foundation
`03-*`	Git collaboration + release workflow	History, branches, merges, tags, recovery
`04-*`	GitLab CI/CD + delivery	`.gitlab-ci.yml`, runners, caches, gates, deploy hooks
`05-*`	Docker + container workflow	Images, Dockerfile, compose, registry, debug
`06-*`	Podman rootless + daemonless posture	Rootless, pods, systemd units, compose compat
`07-*`	Container networking + service comms	Bridges, DNS inside user networks, NAT, proxy shape
`08-*`	Kubernetes orchestration	Workloads, networking in-cluster, storage, ingress, ops
`09-*`	Helm packaging	Charts, values, upgrades, rollbacks
`10-*`	Terraform / IaC lifecycle	State, modules, environments, drift, cloud-ready sketches
`11-*`	Application networking + TLS + foundational proxies	DNS, routing, firewall, host-level Traefik/Nginx drills
`12-*`	Edge reverse proxy + traffic shaping	Compression, TLS termination, LB pools, timeouts, abusive traffic guardrails
`13-*`	Monitoring metrics + Prometheus	Golden signals, scrape model, exporters, PromQL essentials, alerting
`14-*`	Structured logging + Loki aggregation	JSON logs, correlation, central search & forensic discipline
`15-*`	Distributed tracing + OpenTelemetry	Spans propagation, collector, sampling realism
`16-*`	Unified observability + incident investigation hooks	Grafana multi-signal, correlation labs, introductory SLO talk
`17-*`	Secrets & credential lifecycle	env separation, K8s Secret realism, Vault concepts, CI variable hygiene
`18-*`	DevSecOps + supply-chain aware CI	Scanners, SBOM, SAST, pipeline gate storytelling
`19-*`	AWS foundations + operations skim	IAM→VPC→data→edge→ECS/EKS introductions + Terraform/AWS touchpoints ethically
`20-*`	Production reliability rituals	Rollback, backup scepticism, DR tabletop, rollout strategies, blameless retros
`21-*`	Incident troubleshooting scenario labs	K8s, resources, nets, pipelines, datastore timeout narratives
`22-*`	Performance & bottleneck optimisation	Profiler/pprof/cache/pools/load-tests + Postgres/Redis/queue tie-ins
`23-*`	Platform engineering abstraction	Developer platforms golden paths paved roads governance guardrails ethically

Professional depth (**Docker · Kubernetes · Helm · Terraform · AWS**) continues inside those same folders as `11+` / `13+` / `10+` / `13+` / `15+` trailing worksheets—rather than a separate checklist-only area.

Beyond that, specialise only paths your workload demands (**FinOps mastery, SOC2 drills, alternate clouds**).

Source: `01-overview-and-scope/02-understand-implement-break-fix-document-loop.md`

Practise discipline — intentional fault injection and documentation

Rule of the entire path — theory grounded in breakage

Standalone theory consumption is insufficient: for each competency cluster you practise the loop:

Understand the concept → implement it cleanly → deliberately fault the system → **interpret what telemetry and symptoms mean** → repair with controlled reversibility → **document hypotheses, evidence chain, corrective steps, rollback notes**, plus follow-ups that harden recurrence probability downward.

Operational mindset aspiration:

Listen for what the infrastructure is trying to imply — not vague “broken” vibes without inspectable artefacts (logs, service states, resource budgets, timelines).

Source: `01-overview-and-scope/03-required-practice-surfaces-linux-git-phases.md`

Required practice surfaces — Linux stack through platform engineering arcs

Operate only machines and tenants you ethically control.

Linux labs: authorised Ubuntu VMs, disposable directories, scripted reversible failures, reproducible notes (never paste secrets).

Git / GitLab labs: Symfony (or analogous) codebase plus local GitLab (or compliant equivalent) for merge/release rehearsal without customer data leakage.

Downstream corridors (`04` … `23`)

Area	Typical lab ingredients *(adapt to local policy)*
CI/CD GitLab (`04-*`)	`gitlab-runner`, Docker-capable runners, PHPUnit/Composer, PHPStan/static gates, pipeline secret hygiene
Docker (`05-*`)	Docker Engine + Compose plugin + Symfony mocks; trailing units (**`11-*` onward**) deepen **supply-chain & hardening posture** responsibly
Podman (`06-*`)	`podman`, compose compatibility quirks, systemd unit generation
Container networking (`07-*`)	User-defined bridges, internal DNS drills, constrained packet capture tuition
Kubernetes (`08-*`)	Lightweight clusters (**k3d**, kind…), disposable playgrounds; trailing units broaden **capacity, PDB, quotas, NetworkPolicies** ethically
Helm (`09-*`)	Charts on lab clusters; trailing units practise **schemas, hooks/tests, OCI registries, GitOps hand-off language** consciously
Terraform / IaC (`10-*`, `19-*`)	Terraform/OpenTofu + AWS learner accounts responsibly; **`10-*` trailing worksheets** widen backend/apply safety; **`19-*`** extends AWS breadth
App networking foundations (`11-*`)	Host `nginx`, `openssl`, `dig`, `ss`, cautious `tcpdump`
Edge proxies & shaping (`12-*`)	Nginx ↔ Traefik, TLS offload, timeouts, selective compression, sane rate limits
Monitoring (`13-*`)	Prometheus, exporters, PromQL playgrounds, alerting hooks
Logging (`14-*`)	Grafana Loki ingestion stack, correlation IDs across Symfony + Go
Tracing (`15-*`)	OpenTelemetry Collector (+ Tempo/Jaeger-class backends sized for labs)
Observability synthesis (`16-*`)	Grafana boards merging metrics/logs/traces
Secrets (`17-*`)	Vault OSS sandbox optional · GitLab protected variables · K8s Secret realism
DevSecOps (`18-*`)	Trivy, SBOM, SAST/additional scanners in CI
AWS operations (`19-*`)	Core IAM/VPC/compute/storage/ECS/EKS introductions; **`15-*` onward** previews org networking, ECS vs EKS trade space, edges, resilience & cost framing—ethical learner accounts only
Reliability rituals (`20-*`)	Rollbacks, backup/restore rehearsals, rollout strategy narratives
Incident labs (`21-*`)	Controlled chaos across cluster, nets, pipelines, datastore edges
Performance (`22-*`)	Symfony Profiler, Go `pprof`, Redis caches, Postgres plans, synthetic load tooling
Platform engineering arcs (`23-*`)	Golden paths templates summarising repeatable paved roads

Reference materials *(verify listings stay current)*

Track	Surface	Guidance
Linux starter	LinuxJourney	Core modules reinforcing shell/systemd/logs
Linux course	Marketplace “Linux Administration Bootcamp” (titles vary)	Skip enterprise-only tangents unless you branch deliberately
Docker + Kubernetes practical course	Frequently titled **“Docker & Kubernetes: Practical Guide”** style offerings	Fits Docker → networking → Kubernetes → partial Helm scaffolding; deepen AWS/observability here in vault worksheets
K8s exam-prep supplements	KodeKloud CKA flavours / similar	Disposable clusters—not production tenants
Observability backends	Grafana, Prometheus, Loki, OTLP Collector, Jaeger/Tempo docs	Compose minimal stacks; watch resource burn
Scanners	Trivy, optional Syft/Grype, GitLab Ultimate security tiers if licensed	Tune severities thoughtfully
Performance	Symfony Profiler docs, Go `pprof` guide, Postgres performance primers	Cross-link `MySQL-Database-Engineering` parallels for relational debugging habits
AWS	Official Foundations + IAM guides + Well-Architected primer	Honour spend alarms + least-privilege rehearsals

Baseline toolchain hint (Ubuntu-oriented)

After `sudo apt update`, habitual helpers often include `htop`, `tmux`, `jq`, `curl`, `rsync`, `tree`, `vim`; reconcile `net-tools` nostalgia vs `ip` / `ss` modern defaults consciously.

Maintain a lab workspace such as `opslab/` for reproducible scripted exercises and incident notebooks scrubbed of secrets.

Phase — 02-linux-administration-server-mindset

Directory: `02-linux-administration-server-mindset/`

Source: `02-linux-administration-server-mindset/01-filesystem-cli-and-hierarchy-mind-model.md`

Unit 01 — Filesystem posture, hierarchy literacy, habitual CLI navigation

Comfortable primitives: `pwd`, `ls`, `cd`, `mkdir`, `rm`, `mv`, `cp`, `touch`, directory trees (tooling like `tree` when installed), shell history ergonomics (`history`, purposeful clearing rituals — never erase audit trails accidentally on shared infra).

Labs:

```
mkdir -p app logs backup
touch nginx.log db.log
cp nginx.log backup/
mv db.log logs/
```

Mental anchors

Interpret **absolute versus relative paths** consciously when scripting — surprises often trace back ambiguous `cwd`.

Locate standard hierarchy anchors (`/etc`, `/var`, `/tmp`, `/home`, `/usr`, …) and articulate what classes of mutable vs configuration-of-record artefacts typically reside where.

Source: `02-linux-administration-server-mindset/02-users-permissions-privilege-thinking.md`

Unit 02 — Users, ownership, permission bits, audited privilege escalation

Tools: `chmod`, `chown`, `groups`, `sudo`, `whoami`, `id`.

Interpret `-rwxr-xr-x` triplets distinctly for owners, groups, and others—note differing semantics **files vs directories** (execute on directory ≈ traversal / lookup permission).

Controlled experiment:

```
touch secret.txt
chmod 600 secret.txt
chmod 755 deploy.sh   # when ethically present inside disposable lab artefacts
sudo useradd -m testuser   # later remove cleanly with symmetrical housekeeping
```

Mindset emphasis

Incidents originate often from unintended ownership drift, brittle `sudoers` expectations, careless world-readable sensitive paths—practice reading listings like calm pager introspection rehearsals.

Source: `02-linux-administration-server-mindset/03-process-visibility-signalling-and-daemons.md`

Unit 03 — Process visibility, signalling patterns, graceful versus abrupt termination

Comfortable ergonomics: `ps aux`, `top`, `htop`, `kill`, `killall`, job introspection (`jobs`), `bg`, `fg`.

Guided lab

Spawn a deliberately long idle footprint:

```
sleep 500 &
```

Locate PID, validate characteristics in tooling, rehearse escalating signals — prefer cooperative shutdown pathways before abrupt escalation when ethically acceptable locally.

Articulate distinctions between systemd-supervised daemon lifecycles vs ephemeral shell-managed jobs disappearing on session collapse.

Source: `02-linux-administration-server-mindset/04-systemd-and-journal-correlation.md`

Unit 04 — systemd unit literacy plus correlated journalling ingestion

Operational verbs spanning enable/disable choreography, restrained `restart` vs `reload` where documentation differentiates behavioural impact on long-lived sockets.

Inspect aggregates:

```
journalctl -xe
journalctl -u ssh --since "15 min ago"
```

Break-fix rehearsal ethical constraint

Stopping remoting daemons blindly risks lockout — coordinate safeguards (console access, ephemeral VM, reversible snapshots) beforehand.

Source: `02-linux-administration-server-mindset/05-package-management-apt-and-dpkg-thinking.md`

Unit 05 — APT / dpkg ergonomics plus dependency humility

Exercise package lifecycle ethically on ephemeral hosts—for example reversible nginx experimentation when policy aligns; alternatively choose smaller CLI payloads.

Mindful sequences:

```
sudo apt install …
sudo apt remove …
dpkg -l | head                  # illustrative sampling only
```

Mindset rehearsals

Enumerate repository trust sourcing, versioning drift, leftover configuration directories (`purge` ramifications), orphaned dependency fallout, destructive `autoremove` blindness—simulate planning before irreversible housekeeping.

Source: `02-linux-administration-server-mindset/06-ssh-scp-rsync-and-tmux-mindset.md`

Unit 06 — SSH identity, multiplexed sessions, file exchange ergonomics

`ssh-keygen` with conscientious passphrase and agent ergonomics—rehearse sanely inside laboratories reflecting eventual workplace constraints.

Transfers: exploratory `scp` hops plus meticulous `rsync` passes scrutinising traversal semantics (`/` suffix discipline, reciprocal source/destination reciprocity pitfalls).

Operate `tmux` (alternatively classical `screen`) demonstrating detach ↔ reattach resilience against flaky transports—articulate unattended shell pitfalls and jump-host privilege bleed cautions.

Source: `02-linux-administration-server-mindset/07-bash-scripting-automation-habits.md`

Unit 07 — Bash iteration, branching, guard-railed scripting habits

Iteration illustration:

```
#!/usr/bin/env bash
for f in *.log; do
  grep ERROR "$f" || true
done
```

Author `backup.sh` compressing rotations, relocating generations, pruning aged snapshots—with dry rehearsals before trusting destructive trims.

Articulate conscientious quoting, purposeful exit signalling, restrained `errexit` adoption balancing diagnostics friendliness organisational norms prescribe.

Source: `02-linux-administration-server-mindset/08-cron-and-non-interactive-pitfalls.md`

Unit 08 — Cron discipline and silent-environment failure archaeology

`crontab -e` invokes `backup.sh` on a disciplined cadence (example: five-minute iteration only after validating resource impact ethically).

Assume reduced inherited environment semantics—explicit absolute paths, restrained reliance on personalised shell aliases, logging stdout/stderr to inspectable artefacts by design rather than drowning silently inside `MAILTO` oblivion unintentionally configured away.

Source: `02-linux-administration-server-mindset/09-logs-resources-and-guided-fault-remediation.md`

Unit 09 — Correlate logs with disk, sockets, memory, and scheduler artefacts

Fluent use of `tail` / `less`, precise `grep`, `journalctl` windows, **`ss`** (supplement nostalgic `netstat` stories only where legacy estates demand), `df` / `du` / `free`, `uptime` posture.

Controlled fault recipes you author consciously: depleted disk exhaustion, extinguished ancillary unit, brittle permissions starving automation, brittle `cron` path assumptions—recover while narrating reproducible timelines (hypothesis → evidence → corrective diff → safeguards).

Source: `02-linux-administration-server-mindset/10-linux-integration-autonomous-incident-cycle.md`

Unit 10 — Blind rehearsal plus nginx-centric break/recover narration

Navigate without reference crutches initially: correlate symptoms ↔ authoritative logs ↔ measured restarts versus deeper remediation; rectify permission drift hampering scripted operators; reaffirm sane process inventories; scrutinise RAM/disk budgets; dissect scheduler fidelity; reaffirm unaffected SSH ergonomics responsibly.

Laboratory climax: operate nginx—or lighter analogue aligning with organisational policy—and choreograph degrade → recover journaling emphasising deterministic reasoning rather than ad hoc button mashing.

Capture concise artefacts redacting privileged tokens—articulating causal class, illuminating log excerpts corroboratively, enumerated remediation steps plus preventative deltas.

Phase — 03-git-collaboration-and-release-workflow

Directory: `03-git-collaboration-and-release-workflow/`

Source: `03-git-collaboration-and-release-workflow/01-local-git-three-trees-commit-discipline.md`

Unit 01 — Working tree · index · commits

Comfort `git status`, `git diff`, purposeful `git add`, meaningful `git commit`, `git log`, `git restore` choreography.

Laboratory mutate `service.php` (rename sensibly)—inspect diffs stage selectively avoid blind `git add .` habitual damage long-term.

Source: `03-git-collaboration-and-release-workflow/02-branching-and-release-shape.md`

Unit 02 — Parallel development branches

Operate `git branch`, `git switch`; align mental model illustrating `main`↔protective merges↔experiments.

Author disposable feature branches analogous `feature/login`, `bugfix/token` practising naming clarity organisational standards appreciate.

Source: `03-git-collaboration-and-release-workflow/03-merge-and-conflicts.md`

Unit 03 — Merge outcomes and conflict archaeology

Produce deliberate textual conflicts merging divergent snippets—resolve calmly annotating rationale fast-forward versus merge-commit semantics merit conscious distinction.

Source: `03-git-collaboration-and-release-workflow/04-interactive-rebase-history-shape.md`

Unit 04 — Rebasing responsibly

Squash noisy WIP increments via `git rebase -i` practising commit message uplift—never rewrite published shared history recklessly.

Source: `03-git-collaboration-and-release-workflow/05-cherry-pick-revert-reset-recovery-literacy.md`

Unit 05 — Surgical history manoeuvres

Differentiate cherry-picking isolated fixes versus revert-forward-safe undo paths versus destructive reset boundaries—simulate chaos then recover ethically.

Source: `03-git-collaboration-and-release-workflow/06-tags-semver-release-markers.md`

Unit 06 — Annotated tags and semantic intent

Practice lightweight vs annotated tags signalling release anchors—align verbally with semver philosophy teams adopt—even if branching scheme varies.

Source: `03-git-collaboration-and-release-workflow/07-hooks-quality-guardrails.md`

Unit 07 — Client-side hooks (pre-commit quality)

Author minimal hook scaffolding invoking `phpstan` / unit subset—articulate divergence local developer versus CI redundancy intentionally complementary.

Source: `03-git-collaboration-and-release-workflow/08-reflog-safety-net.md`

Unit 08 — Recover from self-inflicted divergence

Leverage `git reflog` after deleting branch mistakenly or hard resetting—articulate TTL awareness garbage collection nuances high-level—not unlimited safety net myths.

Source: `03-git-collaboration-and-release-workflow/09-end-to-end-team-simulation.md`

Unit 09 — Scripted team rehearsal without lookups

Symfony repo choreography: branching, contentious merges, tagging, hypothetical rollback narration—maintain handwritten timeline proving comprehension separate memorised trivia.

Phase — 04-gitlab-cicd-delivery-deployment

Directory: `04-gitlab-cicd-delivery-deployment/`

Source: `04-gitlab-cicd-delivery-deployment/01-first-pipeline-stages-commit-to-result.md`

01 First Pipeline Stages Commit To Result — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Introduce minimalist `.gitlab-ci.yml` illustrating commit→pipeline→job→result narration; align PHPUnit invocation with your Symfony paths.

Source: `04-gitlab-cicd-delivery-deployment/02-gitlab-runner-execution-surfaces.md`

02 Gitlab Runner Execution Surfaces — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Install GitLab Runner; compare docker executor vs shell; rehearse degraded runner/offline/perms/Docker socket failures and recovery.

Source: `04-gitlab-cicd-delivery-deployment/03-cache-and-artifacts-optimisation.md`

03 Cache And Artifacts Optimisation — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Use `cache` (e.g. vendor) and `artifacts` between jobs; practise missing artefact breakage and remediation.

Source: `04-gitlab-cicd-delivery-deployment/04-quality-gates-tests-lint-phpstan.md`

04 Quality Gates Tests Lint Phpstan — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Stages: tests then quality with PHPUnit + lint + PHPStan; inject deliberate PHP and static-analysis faults.

Source: `04-gitlab-cicd-delivery-deployment/05-docker-build-in-pipeline.md`

05 Docker Build In Pipeline — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Pipeline builds image after gates; break Dockerfile expecting failed promotion.

Source: `04-gitlab-cicd-delivery-deployment/06-security-scan-trivy-image.md`

06 Security Scan Trivy Image — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Add image scan step (e.g. Trivy); observe vulnerable-layer reporting and fix path.

Source: `04-gitlab-cicd-delivery-deployment/07-environments-protected-branches.md`

07 Environments Protected Branches — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Define environment naming; block deploy from feature branches; allow controlled promote from protected main.

Source: `04-gitlab-cicd-delivery-deployment/08-deployment-job-automation.md`

08 Deployment Job Automation — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Chain to deploy surrogate (local/stack you own) after scan; narrate lifecycle vs manual babysitting.

Source: `04-gitlab-cicd-delivery-deployment/09-rollback-and-manual-recovery-jobs.md`

09 Rollback And Manual Recovery Jobs — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Deploy bad artefact rehearsal; practise manual rollback / redeploy stable tag discipline.

Source: `04-gitlab-cicd-delivery-deployment/10-full-pipeline-integration-fault-marathon.md`

10 Full Pipeline Integration Fault Marathon — GitLab CI/CD unit

Outcome mindset: Ship software from commit toward production confidently—not YAML memorisation alone.

Practise focus: Full stack rehearsal: push→tests→lint→phpstan→docker build→scan→deploy—rotate faults (runner, lint, Docker, deploy) and document.

Phase — 05-docker-containers-image-workflow

Directory: `05-docker-containers-image-workflow/`

Source: `05-docker-containers-image-workflow/01-images-vs-running-containers.md`

Docker — `01 / images / vs / running / containers`

Focus: Pull nginx, run/list/stop/rm lifecycle; articulate immutable image vs ephemeral writable layer intuition.

Source: `05-docker-containers-image-workflow/02-dockerfile-layer-cache-invalidation.md`

Docker — `02 / dockerfile / layer / cache / invalidation`

Focus: Build PHP/FPM-ish sample altering README vs composer lock observing cache churn discipline.

Source: `05-docker-containers-image-workflow/03-entrypoint-vs-cmd.md`

Docker — `03 / entrypoint / vs / cmd`

Focus: Demonstrate interplay default args vs fixed executable rewriting container argv experiments ethically.

Source: `05-docker-containers-image-workflow/04-volumes-and-bind-mount-persistence.md`

Docker — `04 / volumes / and / bind / mount / persistence`

Focus: Postgres ephemeral loss without named volume versus bind-mount dev ergonomics contrasts.

Source: `05-docker-containers-image-workflow/05-bridge-dns-service-resolution.md`

Docker — `05 / bridge / dns / service / resolution`

Focus: Custom network linking Symfony ↔ Redis ↔ Postgres diagnosing miswired DATABASE_URL.

Source: `05-docker-containers-image-workflow/06-docker-compose-multi-service.md`

Docker — `06 / docker / compose / multi / service`

Focus: Compose stack Symfony+Redis+Postgres+Nginx; environment, restart policy, healthcheck posture.

Source: `05-docker-containers-image-workflow/07-multi-stage-build-optimisation.md`

Docker — `07 / multi / stage / build / optimisation`

Focus: Contrast bloated single-stage Symfony builder vs trimmed runtime artefacts measuring image size deltas.

Source: `05-docker-containers-image-workflow/08-registry-tag-push-pull.md`

Docker — `08 / registry / tag / push / pull`

Focus: Tag, push/pull through registry sandbox articulating versioning promotion story.

Source: `05-docker-containers-image-workflow/09-container-debug-tooling.md`

Docker — `09 / container / debug / tooling`

Focus: Logs, inspect, exec, stats while breaking ports/env/volumes/permissions intentionally.

Source: `05-docker-containers-image-workflow/10-compose-integration-blind-rehearsal.md`

Docker — `10 / compose / integration / blind / rehearsal`

Focus: Stand entire stack sans notes degrade recover documenting lifecycle hypotheses.

Source:
`05-docker-containers-image-workflow/11-image-hardening-nonroot-capabilities-readonly-rootfs.md`

Unit 11 — Image hardening: non-root, capabilities, read-only roots

As images move toward production-facing clusters, predictable UID/GID, dropped Linux capabilities, and read-only container roots materially reduce escalation blast radius—even before admission controllers enforce profiles.

In labs, rebuild a Symfony or Go image so the main process UID is not `0`, mount writable paths explicitly for cache dirs, and iterate `docker inspect` verifying effective user. Pair with Compose override volume permissions consciously rather than widening host permissions lazily.

Document trade-offs: installers expecting root, PHP-FPM user mapping, ephemeral `/tmp`. Capture one rollback story when hardening silently breaks health checks—that narrative is interview-grade depth.

Source: `05-docker-containers-image-workflow/12-buildkit-secrets-and-deterministic-cache.md`

Unit 12 — BuildKit: build secrets vs cache discipline

BuildKit allows mounting short-lived secrets only during build steps, shrinking the chance they land in immutable layers mirrored to registries. Contrast brittle `COPY` of `.env.build` artefacts—acceptable only on disposable workstations under policy you control explicitly.

Practise: convert one multi-stage Dockerfile to use ephemeral secret mounts fetching private Composer/NPM artefacts; quantify cache invalidation triggers when pinning dependency manifests versus stray README tweaks.

Tie scanners (`18-*`) back in: deterministic rebuilds clarify which layer introduced a CVE escalation when base images churn weekly.

Source: `05-docker-containers-image-workflow/13-image-signing-and-verification-consumer-mindset.md`

Unit 13 — Signing & verification as a consumer

You may not operate the organisational signing KMS today, yet you still must articulate how clusters or CI verifies publisher identity: digest references, Sigstore/Cosign-style flows, immutable tags versus floating `latest` regret stories professionally.

Laboratory analogue: practise referencing images by `@sha256:` in Compose or Helm values on disposable clusters verifying identical deploy reproduces—even if organisational signing infra arrives later ethically.

When verification fails mid-pipeline, write the triage headings you would escalate: artefact mismatch, revoked signature, stale trust root rotation narrative responsibly.

Source: `05-docker-containers-image-workflow/14-registry-governance-retention-supply-chain.md`

Unit 14 — Registry policy, retention, and supply-chain cadence

Professional registries differentiate dev/test promotion lanes, immutable release channels, lifecycle policies pruning ancient layers, robotic vulnerability SLA conversations synchronised with base image rotations practiced earlier.

Exercise: emulate two repositories (`dev` / `rel`) tagging strategy; practise garbage-collection awareness (conceptual—not destroying shared org registries unknowingly ethically). Mirror retention policies legal teams sometimes negotiate around personal data artefacts even when absent here consciously.

Leverage backlog hooks: annotate chart (`09-*`) or pipeline (`04-*`) step promoting only signed digests aligning operational compliance peers expect eventually.

Phase — 06-podman-rootless-runtime-model

Directory: `06-podman-rootless-runtime-model/`

Source: `06-podman-rootless-runtime-model/01-daemonless-runtime-basics.md`

Podman — `01-daemonless-runtime-basics`

Focus: Compare Podman daemonless model vs centralized dockerd—lifecycle parity drills.

Source: `06-podman-rootless-runtime-model/02-rootless-namespaces-privilege-mapping.md`

Podman — `02-rootless-namespaces-privilege-mapping`

Focus: Run workloads rootless illustrating user namespace containment vs host-root conflation myths.

Source: `06-podman-rootless-runtime-model/03-pod-multi-container-shared-net.md`

Podman — `03-pod-multi-container-shared-net`

Focus: Group Symfony+sidecars in pod analogue practising shared network namespace ergonomics prepping Kubernetes mental continuity.

Source: `06-podman-rootless-runtime-model/04-registry-push-pull-podman.md`

Podman — `04-registry-push-pull-podman`

Focus: Image tagging/pushing/pull parity with disciplined auth rotation storytelling.

Source: `06-podman-rootless-runtime-model/05-compose-compat-migration.md`

Podman — `05-compose-compat-migration`

Focus: Migrate earlier compose definitions exercising compatibility caveats responsibly.

Source: `06-podman-rootless-runtime-model/06-podman-network-and-volume-drills.md`

Podman — `06-podman-network-and-volume-drills`

Focus: Break DNS, volume mounts, permission skew—repair methodically documenting.

Source: `06-podman-rootless-runtime-model/07-generate-systemd-unit-for-long-running.md`

Podman — `07-generate-systemd-unit-for-long-running`

Focus: Use `podman generate systemd` enabling restart policies surviving reboot ethically on lab VM.

Source: `06-podman-rootless-runtime-model/08-podman-debug-suite.md`

Podman — `08-podman-debug-suite`

Focus: Logs/inspect/exec fault injection narratives bridging Podman quirks.

Source: `06-podman-rootless-runtime-model/09-rootless-stack-integration-rehearsal.md`

Podman — `09-rootless-stack-integration-rehearsal`

Focus: Full Symfony+deps stack rootless degrade/recover without Docker daemon dependency.

Phase — 07-container-networking-service-communication

Directory: `07-container-networking-service-communication/`

Source: `07-container-networking-service-communication/01-bridge-network-isolation-attaching.md`

Container networking unit

Attach services to user-defined bridge isolating workloads; practise detaching dependency causing outages.

Source: `07-container-networking-service-communication/02-internal-dns-hostname-resolution.md`

Container networking unit

Prefer hostname `postgres` / `redis` over brittle hard-coded bridge IPs diagnosing miswired env vars.

Source: `07-container-networking-service-communication/03-nat-and-port-publish-semantics.md`

Container networking unit

Map host `-p host:container` flows versus internal-only ClusterIP-esque thinking bridging future Kubernetes analogies.

Source: `07-container-networking-service-communication/04-localhost-inside-container-antipatterns.md`

Container networking unit

Demonstrate DATABASE_URL redis host pitfalls using `localhost` referencing self not sibling.

Source: `07-container-networking-service-communication/05-overlay-multi-host-concept.md`

Container networking unit

Read-only conceptual skim multi-node overlay prepping cluster networking without premature depth.

Source: `07-container-networking-service-communication/06-nginx-reverse-proxy-ingress-shape.md`

Container networking unit

External client→reverse proxy→app mental model practising broken upstream recovery.

Source: `07-container-networking-service-communication/07-tcpdump-ss-curl-troubleshooting.md`

Container networking unit

Packet/socket introspection ethically on lab traffic isolating DNS, firewall, stale connection symptoms.

Source: `07-container-networking-service-communication/08-stack-integration-dns-nat-proxy.md`

Container networking unit

Raise Symfony+Nginx+Redis+Postgres on custom bridge degrade DNS/NAT/hostnames/ports journaling recoveries.

Phase — 08-kubernetes-orchestration-cluster-lifecycle

Directory: `08-kubernetes-orchestration-cluster-lifecycle/`

Source: `08-kubernetes-orchestration-cluster-lifecycle/01-pod-and-kubectl-foundation.md`

Kubernetes orchestration unit

Pods vs VM confusion avoided; practise kubectl get/describe/logs/exec; corrupt image tug rehearse ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/02-deployment-replica-rollout.md`

Kubernetes orchestration unit

Deployment→ReplicaSet→Pod chain; practise scale, rollout restart, bad image rollback narrative.

Source: `08-kubernetes-orchestration-cluster-lifecycle/03-service-discovery-selectors.md`

Kubernetes orchestration unit

ClusterIP/NodePort; break Service selector mismatches diagnosing DNS stale assumptions.

Source: `08-kubernetes-orchestration-cluster-lifecycle/04-namespace-configmap-secret.md`

Kubernetes orchestration unit

Split dev/stage/prod; mount ConfigMaps vs Secrets cautiously illustrating Symfony JWT secret hygiene redacted journaling.

Source: `08-kubernetes-orchestration-cluster-lifecycle/05-persistentvolume-storage-postgres.md`

Kubernetes orchestration unit

PVC + storage class rehearsal; validate data survives Pod deletion ethically with disposable volumes.

Source: `08-kubernetes-orchestration-cluster-lifecycle/06-ingress-routing-flow.md`

Kubernetes orchestration unit

Ingress bridging edge to Services; degrade rules observe 404/backends mismatch recovery.

Source: `08-kubernetes-orchestration-cluster-lifecycle/07-probes-and-resource-limits.md`

Kubernetes orchestration unit

Liveness/readiness + CPU/mem limits practising overload behavioural differences observably ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/08-horizontal-pod-autoscaling-intro.md`

Kubernetes orchestration unit

HPA synthetic load etiquette only on authorised tiny clusters disclaim resource honesty.

Source: `08-kubernetes-orchestration-cluster-lifecycle/09-rbac-serviceaccount-least-privilege.md`

Kubernetes orchestration unit

Role/RoleBinding exercises denial then calibrated allow reflecting least privilege narratives.

Source: `08-kubernetes-orchestration-cluster-lifecycle/10-affinity-taints-tolerations.md`

Kubernetes orchestration unit

Scheduler steering stories—affinities vs taints/tolerations illustrative within lab constraints acknowledging k3d simplifications.

Source: `08-kubernetes-orchestration-cluster-lifecycle/11-cluster-troubleshooting-correlation.md`

Kubernetes orchestration unit

CrashLoop DNS mounts ingress permission systematic triage scripted fault injections.

Source: `08-kubernetes-orchestration-cluster-lifecycle/12-full-stack-integration-symfony-data-plane.md`

Kubernetes orchestration unit

Symfony+Redis+Postgres+Go+Ingress stacked manifest rehearsal degrade documentation marathon.

Source: `08-kubernetes-orchestration-cluster-lifecycle/13-pod-disruption-budgets-and-controlled-drain.md`

Unit 13 — PodDisruptionBudgets and considerate drains

PDBs translate human availability commitments into scheduler-aware constraints guarding voluntary disruptions (node drains, rollout surges). They do not exempt you from buggy applications—yet they curb thundering herd surprises during upgrades responsibly.

Laboratory story: declare a conservative `maxUnavailable` for a StatefulSet or Deployment handling sessions; combine with `kubectl drain` respecting PDB denials ethically on disposable clusters only conscientiously ethically.

When PDB blocks maintenance, practise communication pattern: quantify risk trade-off, temporary relaxations with peer review—not silent deletion of guarding objects irresponsibly thoughtlessly ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/14-resource-quotas-and-limit-ranges.md`

Unit 14 — Namespace quotas & LimitRanges cooperative fairness

Multi-tenant clusters lean on ResourceQuota enforcing aggregate CPU/mem/object counts plus LimitRanges nudging sane defaults preventing single namespace starving neighbours unknowingly ethically.

Create dev/stage namespaces in k3d: apply quotas, exceed them deliberately reading `kubectl describe quota` frustrations mapping product engineer complaints empathetically for future platform interactions responsibly.

Tie organisational theme: PDB + quota interplay—overly tiny quotas may falsely trigger autoscaler starvation thrash ethically observed micro-lab scale only ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/15-network-policies-layered-micro-segmentation.md`

Unit 15 — NetworkPolicy introductions (Calico/Cilium-class mental model)

NetworkPolicies express allow/deny traffic between Pods and namespaces—implementing coarse micro-segmentation beyond implicit open east-west chatter defaults Kubernetes historically tolerated naively ethically.

When CNI forbids Policies (some minimal labs), simulate mental exercise reading YAML guarding Symfony→Postgres lanes only—not blanket allow-all ethically documented limitations honestly ethically.

Troubleshooting matrix: symmetric label selectors, egress DNS to kube-dns, unintended headless vs cluster IP confusion referencing prior `07-*` foundations bridging consciously ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/16-statefulset-and-volume-patterns-deepening.md`

Unit 16 — StatefulSet operations & resilient storage idioms

Deployments suit ephemeral web tiers; StatefulSets orchestrate replicas with predictable network identities & sticky volumes—for databases or queues you intentionally self-manage ethically documenting risks versus RDS/ElastiCache managed counterparts consciously thoughtfully.

Laboratory rehearsal: Postgres via StatefulSet aligning PVC templates; practise ordered rollout nuances & headless Services DNS entries correlating StatefulSet identity responsibly disposable only ethically thoughtfully.

Surgical upgrade narrative: correlate backup (`20-*`), readiness probes tuning, PDB interplay—simulate controlled failure journaling responsibly ethically.

Source: `08-kubernetes-orchestration-cluster-lifecycle/17-cluster-lifecycle-upgrade-and-deprecation-awareness.md`

Unit 17 — Cluster upgrades & API deprecation vigilance

Professional Kubernetes operators reconcile distributor release notes: removed beta APIs, container runtime transitions, etcd backup cadence—even if partially abstracted managed control planes shift responsibilities sideways not away intentionally consciously.

Maintain personal checklist: staging upgrade rehearsal, conformance smoke tests (`kubectl`-level), Ingress controller compatibility pinning, CSI driver upgrade coupling responsibly ethically conscientiously ethically.

Tie Helm (`09-*`) & Terraform/EKS narratives (`19-*`) when cloud control plane rotates beneath you unexpectedly—coordinate version skew windows consciously consciously ethically conscientiously ethically.

Phase — 09-helm-kubernetes-packaging-deployment

Directory: `09-helm-kubernetes-packaging-deployment/`

Source: `09-helm-kubernetes-packaging-deployment/01-chart-structure-helm-create.md`

Helm packaging unit

Chart.yaml templates/values separation mental model scaffolding first chart ethically.

Source: `09-helm-kubernetes-packaging-deployment/02-values-driven-configuration.md`

Helm packaging unit

Drive replicas image tag ports exclusively via values practising anti-hardcode discipline.

Source: `09-helm-kubernetes-packaging-deployment/03-templates-and-helm-template-render.md`

Helm packaging unit

Templating hygiene using helm template inspecting rendered manifests consciously before apply.

Source: `09-helm-kubernetes-packaging-deployment/04-realistic-symfony-packaging.md`

Helm packaging unit

Bundle Deployment Service ConfigMap Secret illustrating separation of configurable vs confidential.

Source: `09-helm-kubernetes-packaging-deployment/05-chart-dependencies.md`

Helm packaging unit

Upstream dependency charts chaining Redis Postgres responsibly version pinning awareness.

Source: `09-helm-kubernetes-packaging-deployment/06-upgrade-release-lifecycle.md`

Helm packaging unit

Iterative helm upgrade rehearsals narrating immutable release lineage observability minimally.

Source: `09-helm-kubernetes-packaging-deployment/07-rollback-history.md`

Helm packaging unit

Leverage helm history rollback after deliberate misconfiguration appreciating safe regression velocity.

Source: `09-helm-kubernetes-packaging-deployment/08-helm-troubleshooting.md`

Helm packaging unit

Mis-set values/secrets/port skew debugging via helm status get values dry-run mentalities responsibly.

Source: `09-helm-kubernetes-packaging-deployment/09-helm-integration-lab-full-stack.md`

Helm packaging unit

Compose prior lessons across Symfony Go data stores ingress degrade marathon documentation.

Source: `09-helm-kubernetes-packaging-deployment/10-values-schema-validation-and-contracts.md`

Unit 10 — `values.schema.json` and consumer contracts

Charts exploding optional booleans silently produce invalid manifests discovered only runtime—professional charts publish JSON schemas documenting accepted fields and defaults preventing malformed releases early responsibly consciously.

Laboratory adaptation: scaffold minimal schema guarding image repository/tag/replica combos; practise `helm template` failing loudly on purposely invalid CI inputs responsibly ethically conscientiously ethically.

Tie organisational CI: lint chart schema analogous to PHPUnit—fail merges before destructive apply attempts ethically responsibly consciously conscientiously ethically.

Source: `09-helm-kubernetes-packaging-deployment/11-hooks-and-tests-conscious-automation.md`

Unit 11 — Helm hooks & tests harness without surprise outages

Hooks automate pre/post-upgrade jobs (DB migrations)—yet naive hooks extend outage windows unpredictable if jobs wedge mysteriously ethically rehearse TTL & backoff discipline consciously ethically.

Charts may ship tests (`helm test`), mirroring synthetic smoke validations post-install—articulate coupling with Prometheus blackbox exporters vs chart-local job patterns consciously ethically.

Document rollback interplay: aborted hook leaving partial schema mutation demands forward-fix playbook narrative rehearsal responsibly ethically conscientiously ethically.

Source: `09-helm-kubernetes-packaging-deployment/12-helm-oci-registry-and-chart-publishing-flow.md`

Unit 12 — Charts as OCI artefacts

Publishing charts beside images into OCI registries streamlines versioning & RBAC aligning container supply chains—particularly when GitOps controllers fetch charts identically artefacts fetch images ethically documented vendor specifics consciously consciously ethically.

Laboratory analogue: replicate push/pull using authenticated registry sandbox you own ethically; annotate differences versus classic `helm repo index` tarball hosting consciously ethically.

Security interplay: impersonation tokens, robotic CI promotion lanes only after vulnerability scans align `18-*` expectations consciously ethically responsibly consciously ethically.

Source: `09-helm-kubernetes-packaging-deployment/13-gitops-and-release-controller-framing.md`

Unit 13 — GitOps & release-controller mental alignment

Helm often pairs with Flux/Argo CD-class controllers syncing Git-declared releases—professional operators reason about reconcile loops, drift detection, rollback semantics harmonising chart versions with cluster truth consciously ethically conscientiously ethically.

Laboratory light-weight: practise manual Git→`helm upgrade` choreography echoing reconcile cadence—even before adopting full GitOps ethically documenting conceptual mapping consciously ethically.

Pitfalls: conflicting imperative `kubectl` edits battling declarative reconcile loops—establish team rule-of-law consciously consciously ethically ethically.

Phase — 10-terraform-infrastructure-lifecycle

Directory: `10-terraform-infrastructure-lifecycle/`

Source: `10-terraform-infrastructure-lifecycle/01-provider-init-plan-apply.md`

Terraform IaC unit

Declarative model with docker or lightweight provider lab illustrating init/plan/apply destroy discipline ethically disposable resources.

Source: `10-terraform-infrastructure-lifecycle/02-variables-and-outputs.md`

Terraform IaC unit

Parameterising configuration surfacing IPs endpoints via outputs for downstream choreography consciously.

Source: `10-terraform-infrastructure-lifecycle/03-state-and-drift.md`

Terraform IaC unit

Observe terraform state divergence after manual deletes practising refresh reconciliation honesty.

Source: `10-terraform-infrastructure-lifecycle/04-dependency-graph.md`

Terraform IaC unit

Visualise implicit ordering constraints network preceding compute preceding app attachment storytelling.

Source: `10-terraform-infrastructure-lifecycle/05-modules-refactor-duplication-out.md`

Terraform IaC unit

Factor repeated network/compute motifs into reusable module boundaries consciously scoped.

Source: `10-terraform-infrastructure-lifecycle/06-remote-state-and-locking.md`

Terraform IaC unit

Articulate communal state locking preventing concurrent apply clashes referencing S3+Dynamo illustrative reading later AWS phase.

Source: `10-terraform-infrastructure-lifecycle/07-workspaces-or-equivalent-environment-split.md`

Terraform IaC unit

Dev/stage/prod separation strategies noting workspace ergonomics caveats aligning organisational standards.

Source: `10-terraform-infrastructure-lifecycle/08-kubernetes-provider-crossover.md`

Terraform IaC unit

Terraform applying namespace/deployment scaffolding bridging infra vs workload ownership cautiously ethically disposable cluster.

Source: `10-terraform-infrastructure-lifecycle/09-stack-network-vm-k8-integration.md`

Terraform IaC unit

Layered provisioning narrative stitching components respecting dependency graph journaling.

Source: `10-terraform-infrastructure-lifecycle/10-troubleshooting-refresh-and-recovery.md`

Terraform IaC unit

Simulate variable corruption dependency skew recovery via plan discipline documentation.

Source: `10-terraform-infrastructure-lifecycle/11-integration-provisioning-rollout-drill.md`

Terraform IaC unit

Long-form solo rehearsal combining modules outputs remote backends hypothetically ethically.

Source: `10-terraform-infrastructure-lifecycle/12-aws-vpc-ec2-rds-outline.md`

Terraform IaC unit

High-level VPC subnet SG EC2 RDS pattern sketch deferring deep IAM cost optimisation until dedicated cloud track—articulate sequencing honestly reading official guides while practising safely in scratch accounts only.

Source: `10-terraform-infrastructure-lifecycle/13-remote-backend-hardening-and-locking-realities.md`

Unit 13 — Remote backends & locking realism

Teams graduate from local `.tfstate` to remote stores (S3+locking, Terraform Cloud workspaces) guaranteeing collaborative apply coherence—understand SSE encryption, versioning, IAM scoping thoughtfully consciously ethically conscientiously ethically.

Laboratory: configure toy remote backend (ethically segregated sandbox) observing lock contention behaviours when simultaneous applies collide consciously documenting compassionate peer communications consciously ethically ethically.

Pitfalls: orphaned locks after CI abort—articulate unlocking ceremony responsibilities guard-railed with audit consciously ethically ethically.

Source: `10-terraform-infrastructure-lifecycle/14-ci-pipelines-for-plan-and-apply-segregation.md`

Unit 14 — CI segregation: plan vs guarded apply

Professional infra applies split speculative `terraform plan` reviews from narrowed apply roles enforcing branch protections & manual approvals ethically mirroring seriousness production demands consciously ethically.

Compose GitLab/GitHub Actions sketch illustrating OIDC ephemeral credentials rather long-lived IAM user keys ethically emphasising posture improvement narratives consciously ethically.

Discuss blast-radius bounding: targeted `-target` uses sparing emergencies only—not daily habit ethically documenting operational debt consciously ethically.

Source: `10-terraform-infrastructure-lifecycle/15-policy-as-code-basics-and-boundaries.md`

Unit 15 — Policy-as-code (Sentinel / OPA / native guardrails sketch)

Governance attaches automated policy checks rejecting plans violating tagging, encryption toggles, or network openness—articulate interplay with organisational risk appetite without implying one vendor universal ethically consciously ethically.

Laboratory may stub `terraform validate` custom checks simpler first before enterprise policy stacks arrive consciously ethically.

When policy denies legitimate emergency change, escalate human exception workflow consciously consciously ethically ethically.

Source: `10-terraform-infrastructure-lifecycle/16-drift-detection-and-environment-reconciliation.md`

Unit 16 — Drift detection & reconciliation choreography

Declarative infra diverges reality after manual console hotfixes drift detection pipelines surface—professional teams schedule systematic reconciliations or import discipline consciously ethically.

Recreate purposeful drift ethically disposable stack: reconcile via `refresh` narratives & import remediation choices consciously ethically.

Bridging mindset: analogous GitOps reconcile loops share philosophical lineage consciously consciously ethically ethically.

Source: `10-terraform-infrastructure-lifecycle/17-multi-environment-and-module-versioning-strategy.md`

Unit 17 — Multi-environment rollout & semver module pinning

Modules versioned via Git tags/source pins prevent silent surprise upgrades cascading environments—establish promotion pipeline dev→stage→prod with semantic module constraints consciously ethically.

Tie workspaces (`07-*`) or directory-per-env strategies consciously selecting organisational consistent story—not hybrid chaos unconsciously ethically.

Operational note: coupling application Helm releases (`09-*`) with infra module bumps demands coordinated blackout windows conscientiously ethically.

Phase — 11-production-application-network-flow

Directory: `11-production-application-network-flow/`

Source: `11-production-application-network-flow/01-dns-records-and-cidr-literacy.md`

Ops networking traffic unit

dig/nslookup rehearsals plus CIDR addressing vocabulary subnet gateway broadcast clarity.

Source: `11-production-application-network-flow/02-nat-routing-path-traceroute.md`

Ops networking traffic unit

Default gateway traceroute narration articulating asymmetric path misconceptions cautiously ethically.

Source: `11-production-application-network-flow/03-tls-chain-openssl-cli.md`

Ops networking traffic unit

openssl s_client inspection plus nginx TLS misconfiguration breakage recovery lab.

Source: `11-production-application-network-flow/04-nginx-reverse-proxy-upstream-shape.md`

Ops networking traffic unit

Proxy headers upstream selection debugging consciously Symfony upstream analogues.

Source: `11-production-application-network-flow/05-load-balancing-upstream-pool.md`

Ops networking traffic unit

Multi upstream round-robin illustrative combining Go sidecar narrative responsibly.

Source: `11-production-application-network-flow/06-firewall-ufw-thinking.md`

Ops networking traffic unit

Allow/deny rule rehearsal isolating unintended port lockouts consciously recoverable ethically.

Source: `11-production-application-network-flow/07-traefik-dynamic-routes.md`

Ops networking traffic unit

Compose Traefik illustrating automatic router TLS middleware experimentation consciously disposable.

Source: `11-production-application-network-flow/08-cross-layer-troubleshooting-marathon.md`

Ops networking traffic unit

Rotate DNS TLS firewall proxy faults documenting systematic bisection ethically.

Source: `11-production-application-network-flow/09-traefik-nginx-stack-integration.md`

Ops networking traffic unit

Synthetic internet-facing path through proxies to backends plus datastore documenting full trace reasoning.

Phase — 12-edge-reverse-proxy-and-traffic-control

Directory: `12-edge-reverse-proxy-and-traffic-control/`

Source: `12-edge-reverse-proxy-and-traffic-control/01-nginx-forwarding-upstream-shape.md`

Nginx request forwarding sanity

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Topo: browser → nginx → Symfony/Go; probe wrong upstream/port reversibly.
- Iterate proxy_pass ergonomics aligning socket paths responsibly.

Source: `12-edge-reverse-proxy-and-traffic-control/02-forwarding-headers-origin-trust-boundaries.md`

Proxy forwarding headers discipline

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Emit/consume X-Forwarded-For · X-Forwarded-Proto · Host; surface spoofing caveat behind untrusted hops.
- Laravel/Symfony request IP trusting strategy awareness high-level ethically.

Source: `12-edge-reverse-proxy-and-traffic-control/03-tls-termination-edge-to-plain-backend.md`

TLS termination choreography

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Terminate HTTPS outward; sane internal plaintext only within controlled trust envelopes you document honestly.
- `openssl s_client` validate chains; rehearse corrupted cert rollback.

Source: `12-edge-reverse-proxy-and-traffic-control/04-response-compression-gzip-tradeoffs.md`

Compression at reverse proxy tier

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Enable gzip/deflate thoughtfully—measure payload deltas; avoid double-compression nightmares with app layering.

Source: `12-edge-reverse-proxy-and-traffic-control/05-rate-limiting-and-abuse-guardrails.md`

Rate limiting ethos

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Shape limits protecting upstream saturation vs punishing legitimate bursts—simulate flood ethically on lab workloads only.

Source: `12-edge-reverse-proxy-and-traffic-control/06-load-balancing-upstreams-and-health.md`

Upstream pools + availability signalling

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Round-robin illustrative multi Symfony replicas; degrade single member validating resilience narratives.

Source: `12-edge-reverse-proxy-and-traffic-control/07-traefik-dynamic-discovery-middleware.md`

Traefik as modern programmable edge

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Docker/socket providers; HTTPS + middleware chaining (auth trims, redirects) on disposable stacks.

Source: `12-edge-reverse-proxy-and-traffic-control/08-timeouts-buffering-cache-intro.md`

Edge protective timeouts & buffering

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tune proxy ↔ upstream timeouts guarding slow backends; skim cache layers—never cache authenticated surprises naively defaulting.

Source: `12-edge-reverse-proxy-and-traffic-control/09-edge-troubleshooting-matrix.md`

Systematic proxy edge debugging

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Correlate curl, access/error logs, TLS traces, LB target health—fault classes: timeouts, certs, LB 502 ladders.

Source: `12-edge-reverse-proxy-and-traffic-control/10-integration-traefik-nginx-symfony-go-postgres.md`

Full-chain integration rehearsal

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Internet-facing Traefik+Nginx(or single edge choice)—Symfony+Go API+Postgres layering TLS+rates+timeouts; scripted degradation journaling.

Phase — 13-monitoring-metrics-alerting-and-promql

Directory: `13-monitoring-metrics-alerting-and-promql/`

Source: `13-monitoring-metrics-alerting-and-promql/01-golden-signals-and-metric-thinking.md`

What to measure consciously

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Latency · traffic · errors · saturation mental anchors supersetting toy dashboards obsessing meaningless counters.

Source: `13-monitoring-metrics-alerting-and-promql/02-prometheus-scrape-target-lifecycle.md`

Pull-based Prometheus mental model

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Service discovery stubs; scrape interval interplay; practise miswired metrics endpoint diagnosing empty targets ethically.

Source: `13-monitoring-metrics-alerting-and-promql/03-application-metrics-from-symfony.md`

Symfony instrumentation surfacing behaviours

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Expose exemplar histograms/counters practising RED-flavoured stories on synthetic sluggish routes.

Source: `13-monitoring-metrics-alerting-and-promql/04-application-metrics-from-go-runtime.md`

Go instrumentation & runtime vistas

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Process metrics goroutines/mem; saturate goroutine illustrative workload observing ethical ceilings only on lab VMs.

Source: `13-monitoring-metrics-alerting-and-promql/05-node-and-infra-exporters.md`

Bridging bare-metal/container metrics exporters

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Correlate host exporter series with noisy neighbour containers—articulate cardinality caution.

Source: `13-monitoring-metrics-alerting-and-promql/06-label-taxonomy-and-cardinality-awareness.md`

Label discipline versus metric explosions

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Design selective labels sparing high-card combos; practise PromQL slicing safely.

Source: `13-monitoring-metrics-alerting-and-promql/07-promql-essentials-rate-increase-aggr.md`

Foundational PromQL reasoning

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- rate/increase nuances; guarding divide-by-zero naïveté; aggregate windows consciously.

Source: `13-monitoring-metrics-alerting-and-promql/08-alerting-threshold-response-playbooks.md`

Alert design + humane paging philosophy

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Symptom-based vs cause-based alert tension; practise firing synthetic alert on disposable notifier routes only.

Source: `13-monitoring-metrics-alerting-and-promql/09-monitoring-guided-troubleshooting-labs.md`

Metrics-first triage loops

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Inject CPU/mem/latency fault classes map signals → hypotheses succinctly—not instant restart reflex.

Source: `13-monitoring-metrics-alerting-and-promql/10-integration-symfony-go-prometheus-alerts.md`

Stack rehearsal

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tie Symfony+Go scraping + representative alert rules journaling correlation expectations.

Phase — 14-logging-correlation-and-loki-aggregation

Directory: `14-logging-correlation-and-loki-aggregation/`

Source: `14-logging-correlation-and-loki-aggregation/01-log-level-structure-and-context-fields.md`

Deliberate logging payloads

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Structured JSON-ish fields capturing service, severity, durations; avoid screams of useless ALL CAPS strings lacking identifiers.

Source: `14-logging-correlation-and-loki-aggregation/02-structured-application-logs-php-go.md`

Application JSON logging coherence

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Align field naming across Symfony + Go for joinable analytics downstream responsibly.

Source: `14-logging-correlation-and-loki-aggregation/03-correlation-and-request-narratives.md`

Correlation identifiers spanning hops

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Propagate inbound trace/correlation headers across synchronous calls—even before full tracing sophistication matures organically.

Source: `14-logging-correlation-and-loki-aggregation/04-loki-ingestion-retention-thinking.md`

Loki ingestion & slicing queries

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Label hygiene + query performance caution; articulate retention budgeting privacy obligations.

Source: `14-logging-correlation-and-loki-aggregation/05-log-query-and-pattern-mining-discipline.md`

Search tactics isolating anomalies

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Target ERROR/timeouts/auth classes methodically—not infinite scroll burnout.

Source: `14-logging-correlation-and-loki-aggregation/06-layered-infra-vs-app-log-correlation.md`

Which log layer first under ambiguity?

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Tie ingress/proxy/container/app logs sequentially narrating causal chain rehearsal.

Source: `14-logging-correlation-and-loki-aggregation/07-root-cause-narratives-from-log-evidence-alone.md`

RCA insisting on textual evidence artefacts

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Disable instinctive reflex restarts prior comprehension unless safety demands immediate containment ethically.

Source: `14-logging-correlation-and-loki-aggregation/08-integration-centralised-logging-drill.md`

Traefik Symfony Go Redis Postgres → Loki

Outcome mindset: deepen **edge-before-app** reasoning—not stanza-perfect config memorisation alone.

Practise focus

- Unified structured payloads + correlators; scripted failure requiring log-only RCA documentation.

Phase — 15-distributed-tracing-with-opentelemetry

Directory: `15-distributed-tracing-with-opentelemetry/`

Source: `15-distributed-tracing-with-opentelemetry/01-trace-vs-log-vs-metric-splits.md`

When tracing answers *where latency hides*

Outcome mindset: **request narrative** fidelity across cooperating processes.

Practise focus

- Differentiate aggregates (metrics), signal-rich events (logs), segmented timelines (traces).
- Construct minimal multi-span story HTTP→controller→dependency honestly on lab timings.

Source: `15-distributed-tracing-with-opentelemetry/02-opentelemetry-collector-routing-overview.md`

OpenTelemetry ingestion plumbing

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Run collector ethically disposing dev-only exporters—not production secret sprawl defaults.

Source: `15-distributed-tracing-with-opentelemetry/03-instrument-symfony-request-path.md`

Symfony span coverage

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Mark controller vs outbound HTTP spans; induce cooperative slow path observing span elongation ethically.

Source: `15-distributed-tracing-with-opentelemetry/04-instrument-go-service-handlers.md`

Go handler & dependency spans

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Capture DB/external client child spans; amplify SQL latency ethically using disposable dataset only.

Source: `15-distributed-tracing-with-opentelemetry/05-context-propagation-across-php-go.md`

Header propagation choreography

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Guarantee single trace identifiers survive internal hop—not trace disappearance ghost stories blaming vendors simplistically.

Source: `15-distributed-tracing-with-opentelemetry/06-database-span-and-query-latency.md`

Datastore span discipline

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Attach query segments cautiously sanitising literals / PII echoes from statements logging policies.

Source:
`15-distributed-tracing-with-opentelemetry/07-sampling-and-cost-reality-for-tracing-at-scale.md`

Sampling trade-offs

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Head-based vs tail-informed sampling philosophies high-level—not logging every span eternally fantasies realistically.

Source: `15-distributed-tracing-with-opentelemetry/08-span-led-root-cause-rehearsals.md`

Hypothesis narrowing via waterfall spans

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Correlate timeout stories discovering dominating span—not vague slowness hand-waving devoid of timelines.

Source: `15-distributed-tracing-with-opentelemetry/09-integration-multi-hop-tracing-drill.md`

Traefik Symfony Go Redis Postgres stack

Outcome mindset: OTLP-informed **latency archaeology** practising honest propagation.

Practise focus

- Full propagation + deliberate fault requiring trace-led RCA artefacts documented responsibly.

Phase — 16-unified-observability-incidents-and-slo-thinking

Directory: `16-unified-observability-incidents-and-slo-thinking/`

Source: `16-unified-observability-incidents-and-slo-thinking/01-three-pillar-signal-thinking.md`

Three-pillar signal thinking

Outcome mindset: Interpret metrics, logs, and traces as complementary—not interchangeable—during investigations.

Practise focus

- Articulate qualitative roles: compression vs narrative vs segmented latency timelines.
- Dry-run hypothetical API slowdown deciding which pillar you inspect first under ambiguity without dogma.

Source: `16-unified-observability-incidents-and-slo-thinking/02-grafana-multi-signal-dashboards.md`

Unified dashboards with Grafana

Outcome mindset: Bind Prometheus-series visualisation with correlated log/trace drill paths responsibly.

Practise focus

- Fight chart noise; optimise for on-call skim efficiency even in laboratories mirroring seriousness.
- Tie dashboard variables to sane label hygiene preventing accidental cardinal explosions.

Source: `16-unified-observability-incidents-and-slo-thinking/03-cross-signal-correlation-lab.md`

Cross-signal slow dependency laboratory

Outcome mindset: Produce synthetic datastore stall aligning metric rise, textual timeout evidence, elongated SQL-span waterfall.

Practise focus

- Maintain timestamp-aware timeline narrative capturing corroborating artefact excerpts redacted ethically.

Source: `16-unified-observability-incidents-and-slo-thinking/04-incident-investigation-playbook-structure.md`

Structured incident rehearsal

Outcome mindset: Defer restart reflex until hypothesis unless immediate safety containment demands swifter blunt action.

Practise focus

- Capture impact, hypotheses, validations, corrective actions—even for simulated drills.

Source: `16-unified-observability-incidents-and-slo-thinking/05-service-health-using-golden-signals.md`

Comparative service health vistas

Outcome mindset: Observe Symfony versus Go workloads under identical synthetic load ethically scaled to lab realism.

Practise focus

- Explicitly annotate saturation nuances (connections, goroutines, thread pools proxies) verbally.

Source: `16-unified-observability-incidents-and-slo-thinking/06-slo-error-budget-and-sla-awareness.md`

SLO/error budget conversational literacy

Outcome mindset: Discuss availability targets aligning engineering trade-offs vs marketing SLA promises cautiously nuanced.

Practise focus

- Sketch qualitative error budgets without implying financial SLA guarantees absent business context.

Source: `16-unified-observability-incidents-and-slo-thinking/07-deep-span-log-metric-rca.md`

Multi-signal root cause deepening

Outcome mindset: Orchestrate complex fault requiring intertwined telemetry signals isolating bottleneck candidly documenting prevention backlog deltas.

Practise focus

Source: `16-unified-observability-incidents-and-slo-thinking/08-capstone-observability-chaos-integration.md`

Full-stack telemetry integration capstone

Outcome mindset: Traefik Symfony Go Redis Postgres + Prometheus Loki OTel Grafana; rotate chaos degradations journaling succinct postmortem-style artefacts.

Practise focus

Phase — 17-secrets-management-and-credential-security

Directory: `17-secrets-management-and-credential-security/`

Source: `17-secrets-management-and-credential-security/01-classify-production-secrets-vs-config.md`

Classify secrets vs benign config

Outcome mindset: Enumerate credential classes (DB, JWT, cloud keys, TLS private keys) responsibly reviewing Symfony/Go projects without dumping values.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/02-environment-isolation-dotenv-discipline.md`

Environment separation pitfalls

Outcome mindset: Demonstrate staged env separation guarding against leaking prod artefacts into commits or shared clipboards unknowingly.

Practise focus

- Discuss `.env*` layering cautions and secret managers preferred over everlasting root-owned plaintext.

Source: `17-secrets-management-and-credential-security/03-kubernetes-secret-mounting-model.md`

Kubernetes Secret mechanics realism

Outcome mindset: Articulate secrets as encoded-at-rest blobs still requiring RBAC vigilance—not magical encryption fantasies by default unmanaged.

Practise focus

- Mount selectively; forbid logging secret material accidentally during startup debugging.

Source: `17-secrets-management-and-credential-security/04-secret-rotation-drill.md`

Rotation choreography

Outcome mindset: Exercise rotating JWT issuer secret or DB credential sequences observing coordinated rollout dependencies responsibly on labs only.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/05-encryption-boundaries-thinking.md`

At-rest versus in-flight encryption interplay

Outcome mindset: Discuss disk encryption TLS overlays complementing—not replacing—secret hygiene and least privilege thoughtfully.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/06-vault-dynamic-credentials-intro.md`

Dynamic credential issuance concept

Outcome mindset: Understand short-lived leased credentials diminishing static password staleness ethically experimenting with OSS Vault sandbox optionally.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/07-gitlab-ci-secret-variables-policy.md`

CI masking & protected tiers

Outcome mindset: Leverage masked/protected variables; rehearse diagnosing pipeline failures when secret injections missing—avoid printing secrets casually.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/08-credential-incident-remediation-talkthrough.md`

Synthetic leak tabletop

Outcome mindset: Walk tabletop: leaked IAM key revocation ordering, auditing, communicator responsibilities high-level ethically.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `17-secrets-management-and-credential-security/09-integration-secret-delivery-chain-capstone.md`

End-to-end secret delivery narration

Outcome mindset: Compose GitLab protected variables hypothetical · cluster secret artefacts · Symfony Go consumption choreography without plaintext logging.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Phase — 18-devsecops-supply-chain-secure-ci

Directory: `18-devsecops-supply-chain-secure-ci/`

Source: `18-devsecops-supply-chain-secure-ci/01-shift-left-security-culture.md`

Security continuous across lifecycle

Outcome mindset: Articulate integrating checks before deploy—not mythical final gate auditor heroics lone Friday evening.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/02-composer-module-and-go-mod-scans.md`

Dependency scanning ergonomics

Outcome mindset: Run dependency CVE scans on Composer/Go modules; remediate illustrative vulnerable package responsibly using disposable forks.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/03-container-image-scanning-trivy.md`

Registry-bound image scrutiny

Outcome mindset: Pipe built images through Trivy illustrating failing builds on catastrophic base layer CVE severity thresholds consciously humane.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/04-repository-secret-scan-gitleaks-pattern.md`

Secret scanning pipelines

Outcome mindset: Seed synthetic secret purposely removed afterwards verifying scanner prevents promotion—ethical destruction immediately post validation.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/05-sast-baseline-for-php-go.md`

Static analysis augmentation

Outcome mindset: Augment PHPUnit gates with deliberate code smells static analyzers catching—excluding perfectionist noise drowning signal.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/06-sbom-generation-and-review.md`

SBOM transparency

Outcome mindset: Produce bill-of-materials snapshot emphasising organisational traceability confronting supply-chain compromise headlines responsibly.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/07-gitlab-unified-security-pipeline-blocks.md`

Single pipeline quality+security choreography

Outcome mindset: Compose stage ordering: tests · lint/static · dependency/image/secret scanners · build · deploy ethically pausing merges on regressions thoughtfully.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/08-supply-chain-threat-model-talkthrough.md`

High-level attacker narrative arcs

Outcome mindset: Enumerate dependency poisoning, compromised base images, exposed pipeline credentials—without fearmongering absolving engineering laziness casually.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/09-incident-remediation-supply-chain-mindset.md`

Security incident rehearsal

Outcome mindset: Conduct tabletop on scanner bypass or leaked pipeline token—articulate revocation + rotation discipline succinctly ethically.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Source: `18-devsecops-supply-chain-secure-ci/10-integration-devsecops-capstone-gitlab-docker.md`

Integrated security gate capstone

Outcome mindset: Unify Symfony+Go Dockerfile pipeline with scanners toggled intentionally failing remediation loops documenting deltas responsibly.

Practise focus

(Capture lab evidence in notebooks you secure; omit secrets verbatim.)

Phase — 19-aws-cloud-infrastructure-and-operations

Directory: `19-aws-cloud-infrastructure-and-operations/`

Source: `19-aws-cloud-infrastructure-and-operations/01-iam-identities-policies-least-privilege.md`

IAM fundamentals + least-privilege rehearsals

Outcome mindset: IAM fundamentals + least-privilege rehearsals

Practise focus

- Enumerate identity vs permission vs resource mapping; revoke-over-grant tabletop ethically on scratch IAM users disposable.

Source: `19-aws-cloud-infrastructure-and-operations/02-ec2-security-groups-instance-access.md`

EC2 compute introduction

Outcome mindset: EC2 compute introduction

Practise focus

- Bootstrap Ubuntu instance practising restricted Security Group deltas; articulate key hygiene vs ephemeral session managers high-level ethically.

Source: `19-aws-cloud-infrastructure-and-operations/03-vpc-subnets-and-routing.md`

VPC topology literacy

Outcome mindset: VPC topology literacy

Practise focus

- Contrast public/private subnet roles, IGW versus NAT gateways, route table debugging narratives responsibly cost-aware ethically.

Source: `19-aws-cloud-infrastructure-and-operations/04-rds-managed-postgresql-access-shape.md`

Managed relational posture

Outcome mindset: Managed relational posture

Practise focus

- Exercise SG-restricted DB connectivity from app tier; forbid wide-open Postgres anti-pattern horror stories ethically.

Source: `19-aws-cloud-infrastructure-and-operations/05-s3-object-storage-and-policies.md`

S3 ergonomics distinguishing object semantics

Outcome mindset: S3 ergonomics distinguishing object semantics

Practise focus

- Demonstrate versioning/lifecycle interplay cautiously aligning compliance unknowns disclaimers ethically.

Source: `19-aws-cloud-infrastructure-and-operations/06-route53-alb-and-target-health.md`

DNS pointing at load-balanced targets

Outcome mindset: DNS pointing at load-balanced targets

Practise focus

- Articulate layered health probing responsibilities across ALB target groups ethically rehearsing outage simulations disposable environments.

Source: `19-aws-cloud-infrastructure-and-operations/07-cloudwatch-metrics-and-alarms.md`

CloudWatch grounding

Outcome mindset: CloudWatch grounding

Practise focus

- Hook CPU-oriented synthetic alarm illustrative only—avoid blasting production paging endpoints inadvertently ethically.

Source: `19-aws-cloud-infrastructure-and-operations/08-cloud-secret-providers-intro.md`

Cloud secret distribution sketch

Outcome mindset: Cloud secret distribution sketch

Practise focus

- Discuss AWS secret manager injecting runtime credentials aligning rotation responsibilities organisationally ethically.

Source: `19-aws-cloud-infrastructure-and-operations/09-ecs-services-and-task-networking.md`

ECS container orchestration primer

Outcome mindset: ECS container orchestration primer

Practise focus

- Contrast task definitions vs services; practise tiny deploy illustrative acknowledging Fargate/EC2 trade depth deferred consciously.

Source: `19-aws-cloud-infrastructure-and-operations/10-eks-managed-kubernetes-bridge.md`

Managed Kubernetes bridging prior k3d

Outcome mindset: Managed Kubernetes bridging prior k3d

Practise focus

- EKS mental overlay referencing earlier Helm/Terraform curricula cross-linking ethically cost cautiously.

Source: `19-aws-cloud-infrastructure-and-operations/11-iam-policy-debug-security-tabletop.md`

Policy debugging tabletop

Outcome mindset: Policy debugging tabletop

Practise focus

- Exercise overly tight IAM provoking pipeline breakage diagnosing CloudTrail-esque audit signals high-level ethically read-only rehearsals.

Source: `19-aws-cloud-infrastructure-and-operations/12-lambda-serverless-fit-sketch.md`

Serverless applicability nuance shallow

Outcome mindset: Serverless applicability nuance shallow

Practise focus

- Demonstrate miniature function handler emphasising operational limits—not universal hammer fantasy ethically.

Source: `19-aws-cloud-infrastructure-and-operations/13-terraform-aws-topology-capstone-recap.md`

Terraform choreography revisiting IaC

Outcome mindset: Terraform choreography revisiting IaC

Practise focus

- Praise reproducible infra vs console-only drift using Terraform snippets responsibly disposable accounts only ethically.

Source: `19-aws-cloud-infrastructure-and-operations/14-private-tier-connectivity-deepening.md`

Private tier connectivity deepening

Outcome mindset: Private tier connectivity deepening

Practise focus

- Discuss ALB bridging internet to private subnets housing apps + RDS ethically sketching segmentation responsibilities explicitly.

Source: `19-aws-cloud-infrastructure-and-operations/15-aws-organizations-scps-and-cross-account-access.md`

Unit 15 — AWS Organizations, SCPs, and cross-account access patterns

Large AWS estates centralise guardrails through AWS Organizations and **Service Control Policies** (SCPs) constraining what member accounts can ever enable—even if an IAM policy inside the account looks permissive on paper. This is not `19-*` day-one material, but you must recognise the existence of control planes above single-account IAM when designing landing zones.

Pair SCP thinking with **cross-account IAM roles** (typically `sts:AssumeRole` from a shared tooling account) so CI/CD or human break-glass never stores long-lived access keys on laptops. Sketch a three-account mental model: `security`, `workloads`, `network`—even if your lab only simulates one account.

Verification habit: when a pipeline suddenly cannot create an S3 bucket, consider **implicit deny via SCP** before spending hours debugging IAM inline policies in isolation.

Source: `19-aws-cloud-infrastructure-and-operations/16-transit-gateway-and-multi-vpc-network-topology.md`

Unit 16 — Transit Gateway style multi-VPC connectivity (conceptual lab)

When services outgrow a single VPC, teams connect spoke VPCs through **AWS Transit Gateway** (or peering patterns for smaller footprints) so shared services (egress inspection, centralised logging, private API endpoints) remain reachable without duplicating NAT infrastructure everywhere.

You may not build a full TGW lab on Free Tier; still draw address plans showing **non-overlapping RFC1918 CIDRs**, route table priorities, and where **inspection VPCs** live. Compare mental model to Kubernetes overlay networking (`07-*`, `08-*`).

Failure mode to rehearse narratively: asymmetric routing after adding a new spoke—document how you would prove with VPC Flow Logs (conceptual) rather than guessing security groups first.

Source: `19-aws-cloud-infrastructure-and-operations/17-ecs-fargate-vs-eks-operational-trade-space.md`

Unit 17 — ECS/Fargate vs EKS: choosing a container platform on AWS

Amazon ECS (often with Fargate) packages task definitions, service discovery, and scaling with less moving parts than **EKS**, which brings the full Kubernetes operational surface you already practised in `08-*`/`09-*`. Neither is universally superior: choose based on team skills, desired extension points, and integration with existing GitOps stacks.

Exercise: document a decision matrix for a Symfony + Go API stack: who owns cluster upgrades, how Helm charts port, what observability agents you must run, and how CI promotes images. Even a one-page matrix is enough to prove structured thinking.

Cost lens: Fargate removes node management but can surprise at steady high CPU—pair with cost dashboards later (`20-*` reliability themes) before production promises.

Source: `19-aws-cloud-infrastructure-and-operations/18-edge-security-cloudfront-waf-and-acm-practices.md`

Unit 18 — Edge delivery: CloudFront, WAF, and ACM in practice

Public HTTP surfaces often terminate TLS at **CloudFront** or **Application Load Balancers** with certificates from **ACM**. Professional operators understand cache key behaviours, origin shielding, and when to enable **AWS WAF** managed rule groups versus custom rules—without enabling every rule aggressively blocking legitimate API clients.

Lab substitute if budget-constrained: keep using `12-*` Traefik/Nginx locally but map each feature (TLS cert auto-renewal, geo headers, request logging) to its AWS analogue in writing.

Incident tie-in: when users see sporadic 403s, differentiate WAF vs security group vs origin misconfiguration using edge access logs (conceptual workflow).

Source: `19-aws-cloud-infrastructure-and-operations/19-resilience-backup-and-dr-patterns-on-aws.md`

Unit 19 — Resilience, backup, and DR patterns (RDS, S3, snapshots)

Translate `20-*` reliability ideas into AWS primitives: automated **RDS snapshots** with tested restores, **S3 versioning** + lifecycle rules, **cross-region replication** only when RPO demands justify cost. Always script a restore rehearsal summary—even if the rehearsal happens quarterly in a real job.

Document RPO/RTO targets qualitatively for a demo ecommerce database: how many minutes of data loss are tolerable, which snapshot window satisfies that, and who approves emergency PITR spends.

Pair with Terraform (`10-*`) cautiously: removing `DeletionProtection` via IaC is a production foot-gun—use policy checks introduced in `10-*` trailing units.

Source: `19-aws-cloud-infrastructure-and-operations/20-cost-governance-tagging-and-quotas.md`

Unit 20 — Cost awareness, tagging, and service quotas

Professional AWS work includes **cost allocation tags**, **budgets + anomaly detection**, and understanding **service quotas** before large-scale load tests take down an API Gateway unexpectedly. This is not FinOps certification depth—just enough to partner credibly with finance peers.

Exercise: define a mandatory tag schema (`Environment`, `Owner`, `CostCenter`) you would enforce via SCP or IaC validators. Describe how Grafana/Prometheus billing exporters differ from AWS Cost Explorer (both useful, different fidelity).

When scaling EKS (`10-eks*` narratives), remind yourself control plane hourly charges plus NAT Gateway data processing often dominate guesses—surfacing early in architectures avoids shock.

Source: `19-aws-cloud-infrastructure-and-operations/21-hybrid-connectivity-vpn-and-direct-connect-overview.md`

Unit 21 — Hybrid connectivity: Site-to-Site VPN & Direct Connect concepts

Many enterprises attach AWS to on-prem Active Directory realms or legacy mainframes via **Site-to-Site VPN** tunnels or leased-line style **Direct Connect**. You may never provision DX in a learner account, yet architecture reviews demand vocabulary: BGP prefixes, failover tunnels, resilience expectations.

Laboratory alternative: emulate VPN concepts with overlapping route scenarios in diagrams, then correlate to Kubernetes clusters needing outbound access to corp HTTP proxies responsibly.

Troubleshooting pattern: asymmetric routing after corporate firewall updates—coordinate with netops using traceroute artefacts from earlier Linux (`02-*`) and container networking drills (`07-*`).

Phase — 20-production-reliability-and-release-safety

Directory: `20-production-reliability-and-release-safety/`

Source: `20-production-reliability-and-release-safety/01-rollbacks-across-deploy-surfaces.md`

Rollback choreography across Helm, GitLab, and runtime toggles

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Contrast redeploy previous artefact vs forward-fix strategies responsibly articulating downtime trade-offs ethically.

Source: `20-production-reliability-and-release-safety/02-database-backups-and-restore-validation.md`

Backup scepticism pipeline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Operate Postgres backup rehearsal emphasising restores-not-backups existential lesson responsibly destroying disposable dataset only ethically.

Source: `20-production-reliability-and-release-safety/03-disaster-recovery-tabletop-shape.md`

Disaster recovery narrative arcs

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Author concise recovery objectives RTO/RPO vocabulary qualitatively without enterprise consulting cosplay overshadowing practicality ethically.

Source: `20-production-reliability-and-release-safety/04-blue-green-rollout-shape.md`

Blue/green parallelism mental model

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Sketch parallel environment promotion toggling verifying health gates before draining old colour ethically on lab mocks only.

Source: `20-production-reliability-and-release-safety/05-canary-and-progressive-traffic-shift.md`

Canary / progressive exposure

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Discuss partial traffic bifurcation monitoring error budgets qualitatively referencing service mesh proxies or Ingress weighting ethically cautiously nuanced.

Source: `20-production-reliability-and-release-safety/06-postmortem-format-and-blameless-accountability.md`

Postmortem structure

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Produce incident retrospective template outlining impact timelines root causal factors preventative backlog accountability responsibly.

Source: `20-production-reliability-and-release-safety/07-slo-sla-awareness-for-operators.md`

SLO versus marketing SLA distinctions

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Converse meaning of availability targets thoughtfully acknowledging customer promises vs engineer error budget nuance ethically without numeric absolutism absent data.

Phase — 21-incident-troubleshooting-scenario-labs

Directory: `21-incident-troubleshooting-scenario-labs/`

Source: `21-incident-troubleshooting-scenario-labs/01-kubernetes-database-visibility-scenarios.md`

Kubernetes cannot reach database investigations

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Trace Service selector omissions, DNS flake, NetworkPolicy omission, Secrets/Config discrepancies ethically simulating disruptions disposable clusters responsibly.

Source: `21-incident-troubleshooting-scenario-labs/02-pod-instability-crashloops-resource-pressure.md`

Pod restart storms

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Correlate OOMKill vs failing readiness probes vs missing Secret volumes ethically rehearsing kubectl describe narratives responsibly.

Source: `21-incident-troubleshooting-scenario-labs/03-hot-cpu-hot-memory-metrics-led-triage.md`

Resource contention triage loops

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Refuse instantaneous restart urge; escalate through metrics/logs/traces alignment ethically synthetic load generation labs only responsibly.

Source: `21-incident-troubleshooting-scenario-labs/04-networking-tls-dns-ingress-502-chains.md`

Edge + mesh networking blackout rehearsals

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Practise TLS chain validation, DNS resolution failures, Ingress misroutes producing 502 ladders ethically documenting hypotheses responsibly.

Source: `21-incident-troubleshooting-scenario-labs/05-gitlab-runner-and-image-pipeline-incidents.md`

Pipeline failure forensics

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Diagnose brittle secrets, degraded runners, Dockerfile cache regressions ethically resisting blind rebuild loops sixfold irresponsibly.

Source: `21-incident-troubleshooting-scenario-labs/06-database-and-cache-timeout-scenarios.md`

Datastore latency incident simulations

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Correlate connection pool starvation vs slow queries vs saturation misconfiguration responsibly synthetic fault injections disposable databases ethically.

Source: `21-incident-troubleshooting-scenario-labs/07-full-stack-chaos-integration-doc.md`

Full stack chaos journaling

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Compose week synthesising rotational faults demanding structured evidence→hypothesis→mitigation artefacts responsibly ethically.

Phase — 22-performance-bottleneck-and-runtime-optimization

Directory: `22-performance-bottleneck-and-runtime-optimization/`

Source: `22-performance-bottleneck-and-runtime-optimization/01-performance-instrumentation-before-optimisation.md`

Measure-before-change discipline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Enumerate latency vs throughput distinctions emphasising forbidding optimisation theatre absent baselines ethically.

Source: `22-performance-bottleneck-and-runtime-optimization/02-symfony-profiler-query-explosion-awareness.md`

Symfony Profiler & Doctrine query counts

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Demonstrate hypothetical N+1 detection remediation responsibly measuring before/after query counts ethically on disposable fixtures.

Source: `22-performance-bottleneck-and-runtime-optimization/03-go-pprof-cpu-memory-goroutines.md`

Go profiling surfaces

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Operate pprof ethically capturing goroutine/leak illustrative workloads annihilated afterwards responsibly avoiding production surprises.

Source:
`22-performance-bottleneck-and-runtime-optimization/04-redis-caching-tradeoffs-invalidations.md`

Redis cache coherence discipline

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Articulate TTL invalidation pitfalls cache stampede mitigations ethically synthetic load glimpses labs only responsibly.

Source:
`22-performance-bottleneck-and-runtime-optimization/05-database-connection-pool-tuning-shape.md`

Connection pool starvation narratives

Outcome mindset: reproducible resilience rituals—not folklore heroism alone.

Practise focus

- Tune Postgres pool boundaries observing exhaustion timeouts ethically restrained concurrency ramps laboratories responsibly.

Source: `22-performance-bottleneck-and-runtime-optimization/06-postgresql-slow-query-analysis.md`

Postgres latency & planning awareness

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Use EXPLAIN/ANALYZE patterns on disposable data; correlate with application query shapes responsibly.

Source: `22-performance-bottleneck-and-runtime-optimization/07-synthetic-load-k6-or-apachebench.md`

Load testing introductions

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Ramp concurrency ethically on lab stacks; observe p95 latency + error interplay without DDoSing shared networks.

Source: `22-performance-bottleneck-and-runtime-optimization/08-integration-bottleneck-hunt-marathon.md`

Cross-signal bottleneck capstone lab

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Combine profiler hints, Prometheus series, spans to isolate dominating constraint ethically rotating fault injections.

Source: `22-performance-bottleneck-and-runtime-optimization/09-postgres-operations-vacuum-and-stats-sketch.md`

Operations extensions for Postgres operators

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Touch autovacuum & pg_stat_statements narratives at high altitude deferring encyclopaedic mastery consciously.

Source:
`22-performance-bottleneck-and-runtime-optimization/10-redis-production-operational-cautions.md`

Redis memory/eviction operational literacy

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Discuss persistence modes thundering herds mitigation responsibly illustrative synthetic mis-TTL rehearsals reversible ethically.

Source: `22-performance-bottleneck-and-runtime-optimization/11-async-queue-shape-and-worker-failure.md`

Queue systems failure literacy

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Outline DLQ retries idempotency using RabbitMQ/redis-stream parallels responsibly high-level illustrative labs ethically.

Source: `22-performance-bottleneck-and-runtime-optimization/12-database-centric-incident-scenarios-from-performance-view.md`

DB incidents bridging ops performance curricula

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Simulate deadlock/lock contention exhaustion tying metrics/logs/traces ethically disposable databases only conscientiously conscientiously ethically.

Source: `22-performance-bottleneck-and-runtime-optimization/13-async-queue-incident-drills.md`

Queue backlog escalation investigations

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Observe stalled consumers backlog growth alerting loops responsibly ethically lab poison messages eradicated afterwards thoughtfully.

Source:
`22-performance-bottleneck-and-runtime-optimization/14-performance-capstone-integrated-stack.md`

Performance optimisation capstone rehearsal

Outcome mindset: evidence-led optimisation—not cargo-cult caches.

Practise focus

- Traefik Symfony Go Redis Postgres Prometheus stack degrade→measure→repair journaling responsibly ethically.

Phase — 23-platform-engineering-and-internal-developer-platforms

Directory: `23-platform-engineering-and-internal-developer-platforms/`

Source: `23-platform-engineering-and-internal-developer-platforms/01-internal-developer-platform-problem-shape.md`

Why platform teams mitigate ticket queues

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Articulate alleviating fragmented bespoke infra setups duplicating latent risk exponentially organisationally ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/02-golden-paths-vs-chaos-custom-deploys.md`

Standard paved roads coexist with bounded flexibility

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Contrast fifty snowflake deployments vs opinionated templated corridors lowering cognitive load ethically thoughtful governance interplay.

Source: `23-platform-engineering-and-internal-developer-platforms/03-self-service-provisioning-fiction-to-practice.md`

Self-service infra narrative responsibly incremental

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Sketch GitLab-triggered provisioning reducing wait-time ethically acknowledging guardrails prerequisites observability hooks mandatory ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/04-banish-eternal-console-clickops-default.md`

Automation default posture

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Advocate IaC/GitOps pipelines superseding brittle manual consoles except rare break-glass situations documented ethically conscientiously ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/05-composable-modules-pipeline-templates.md`

Reusable modules & Helm/Terraform scaffolding

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Package network/compute Helm baseline templates ethically harmonising divergence consciously governance reviewed responsibly.

Source: `23-platform-engineering-and-internal-developer-platforms/06-service-catalog-abstraction-shape.md`

Service catalogue thinking

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Enumerate shared datastore cache queue secrets telemetry bundles developer selects responsibly reducing bespoke snowflakes ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/07-platform-governance-guardrails-quality-bars.md`

Governance marrying enablement plus control

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Mandate telemetry security scanning ownership metadata baseline before production eligibility ethically harmonising organisational policy convergence responsibly.

Source: `23-platform-engineering-and-internal-developer-platforms/08-integration-paved-road-capstone-overview.md`

Paved-road integration storytelling

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Compose GitLab Dockerfile Helm Terraform Observability arcs minimising repetitive manual scaffolding ethically narrative capstone ethically deliberately conscientiously ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/09-feedback-loops-platform-and-product.md`

Operational feedback bridging platform & product engineering

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Discuss SLO-informed prioritisation looping platform debt versus feature velocity tensions responsibly qualitatively ethically.

Source: `23-platform-engineering-and-internal-developer-platforms/10-deepening-reading-when-ready.md`

Defer deeper specialisation responsibly

Outcome mindset: scalable developer leverage—not buzzword frosting alone.

Practise focus

- Pointer: revisit AWS/advanced Postgres/queues after practising depths earlier areas emphasising iterative deepening ethically consciously.